



Activity 10: Banking Operations with Abstract Accounts



Objective

In the previous activity, you created an `AbstractAccount` and learned how different account types can follow different withdrawal rules.

Now let's use those accounts to build a simple **Banking Operations Engine**.

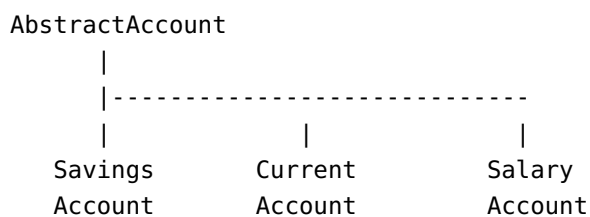
In this activity, you will practice:

- Abstract classes
- Inheritance
- Polymorphism
- Arrays of parent-class objects
- Method overriding
- Exception handling
- Fund transfer between accounts
- Type checking using `instanceof`



Your Task

You are given the following account classes:



You need to complete only:

```
src/com/gdb/tests/TestAbstractAccount.java
```



Do not modify the account classes for this activity.

Your job is to write the test/driver program that uses the existing classes.

Part 1 – Create an Account Portfolio

A bank can have different types of accounts.

Create an array of:

```
AbstractAccount[]
```

and store different account objects inside it.

For example, your portfolio should contain:

- One `SavingsAccount`
- One `CurrentAccount`
- One `SalaryAccount`

Think about this

Even though these are different classes, they all inherit from:

```
AbstractAccount
```

So can you store all of them in a single array?

 Try it!

Concept:

```
AbstractAccount[]  
|  
| --- SavingsAccount  
| --- CurrentAccount  
| --- SalaryAccount
```

This is an example of **polymorphism**.

Part 2 – Create a Fund Transfer Operation

Now imagine a customer wants to transfer money from one account to another.

For example:

Savings Account
Balance = Rs 10000

|
| Transfer Rs 3000
↓

Current Account
Balance = Rs 5000

After the transfer:

Savings Account → Rs 7000
Current Account → Rs 8000

Your Task

Create a method such as:

```
transfer(sourceAccount, destinationAccount, amount, pin)
```

The method should perform the following steps.

Step 1 – Withdraw from the source

Use the source account's existing:

```
withdraw()
```

method.

Pass:

- Transfer amount
- Source account PIN

Remember that `withdraw()` already performs validations such as PIN, account status, amount, and account-specific debit rules.

Step 2 – Deposit into destination

Only if the withdrawal is successful, deposit the same amount into the destination account.

For example:

Transfer = Rs 3000

Source:

Rs 10000 → Rs 7000

Destination:

Rs 5000 → Rs 8000



Important: Handle Failed Transfers

What should happen if the withdrawal fails?

For example, suppose the customer enters the wrong PIN.

Source Balance = Rs 10000

Destination Balance = Rs 5000

Wrong PIN ❌

The transfer should fail.

Most importantly:

! The destination account must NOT receive the money.

The balances should remain:

Source = Rs 10000

Destination = Rs 5000



Think about the order carefully

Should you:

deposit → withdraw

or:

withdraw → deposit

?

Think about what would happen if the withdrawal fails.

Part 3 – Handle Exceptions

Your transfer method should handle exceptions gracefully.

Possible exceptions can come from the withdrawal operation.

For example:

Wrong PIN

```
InvalidPinException
```

Inactive Account

```
InactiveAccountException
```

Invalid Amount

```
InvalidAmountException
```

Insufficient Balance / Account Rule

Depending on the account type, an exception may be thrown if the withdrawal is not allowed.

Your program should catch the exception and display a meaningful message instead of allowing the program to terminate unexpectedly.



Test Case 1 – Successful Transfer

Create accounts with suitable balances.

Example:

```
Savings Account Balance = Rs 10000  
Current Account Balance = Rs 5000
```

Transfer:

Rs 3000

using the correct Savings Account PIN.

Expected result

Transfer Rs 3000 from Savings to Current: SUCCESS

Savings Balance: Rs 7000.0

Current Balance: Rs 8000.0

✓ Both balances should change correctly.

Test Case 2 – Failed Transfer Due to Wrong PIN

Now try another transfer using an incorrect PIN.

For example:

Savings Balance = Rs 7000

Current Balance = Rs 8000

Transfer = Rs 2000

PIN = WRONG

The transfer should fail.

Expected result

Failed Transfer (Wrong PIN):
Exception caught, no balance changed [PASS]

After the failed transfer:

Savings Balance = Rs 7000

Current Balance = Rs 8000

★ Important

The destination account must **not** be credited when the source withdrawal fails.

Part 4 – Process the Monthly Banking Cycle

Now imagine that the bank performs some monthly maintenance.

You already have an array containing different account types:

```
AbstractAccount[]
```

Loop through the array.

For each account, determine what type of account it is.



Savings Account – Apply Interest

If the account is a:

```
SavingsAccount
```

apply its interest.

The Savings Account class already provides:

```
applyInterest()
```

which calculates interest based on the current balance and interest rate.

Example

Suppose:

```
Savings Balance = Rs 10000  
Interest Rate = 4%
```

After applying interest:

```
Interest = Rs 400  
  
New Balance = Rs 10400
```

Your task is to identify the Savings Account in the portfolio and call:

```
applyInterest()
```

Salary Account – Check Salary Activity

If the account is a:

```
SalaryAccount
```

you need to check its salary activity.

The class contains:

```
inactiveMonths
```

and provides:

```
incrementInactiveMonths()
```

to update the value.

For this activity, demonstrate that your monthly cycle can identify a Salary Account and process its salary-related information.

Think about:

```
Is this account a SalaryAccount?  
  |  
  YES  
  |  
  Process salary account activity
```

Key Concept – instanceof

You will need to identify the actual account type stored inside the `AbstractAccount[]`.

For example, think about:

```
if account is SavingsAccount  
  → apply interest
```

```
if account is SalaryAccount
    → process salary activity
```

Java provides the `instanceof` operator to help you determine the actual object type.

Example idea

```
AbstractAccount account
```

↓

```
Is it a SavingsAccount?
```

↓ YES

```
Apply interest
```

Try to write this yourself.

Think Before You Code

Before starting the implementation, answer these questions:

Question 1

Why can we create:

```
AbstractAccount[]
```

and store:

```
SavingsAccount
CurrentAccount
SalaryAccount
```

inside it?

Question 2

Which method should be used to withdraw money?

Look at the methods available in `AbstractAccount`.

Question 3

If withdrawal fails because of an incorrect PIN, should you deposit money into the destination?

👉 Why?

Question 4

How can you identify whether an object in the array is a `SavingsAccount`?

Question 5

Which method should you call to apply Savings Account interest?

Look at the `SavingsAccount` class.

Question 6

Which methods/fields can you use to process Salary Account activity?

Look at the `SalaryAccount` class.

Suggested Test Scenarios

Your program should test at least these scenarios:

Test 1 – Successful Transfer

```
Savings → Current  
Amount = Rs 3000  
Correct PIN
```

Expected:

```
Transfer successful  
Savings Balance: Rs 7000.0  
Current Balance: Rs 8000.0
```

Test 2 – Wrong PIN

```
Savings → Current  
Amount = Rs 2000  
Wrong PIN
```

Expected:

```
Exception caught  
No balance changed
```

Test 3 – Monthly Savings Interest

Create a Savings Account and process the monthly cycle.

Expected:

```
Savings interest applied successfully
```

The balance should increase according to the account's interest rate.

Test 4 – Salary Account Processing

The monthly cycle should identify the Salary Account and perform the required salary activity.

Expected:

```
Salary account monthly processing completed
```

★ Expected Output

Your final program should produce output similar to:

```
=== Activity 10: Banking Operations Suite ===  
  
Transfer Rs 3000 from Savings to Current: SUCCESS  
Savings Balance: Rs 7000.0 | Current Balance: Rs 8000.0  
  
Failed Transfer (Wrong PIN): Exception caught, no balance changed [PASS]
```

Monthly Interest Cycle processed for all qualifying accounts.

All banking operations passed!

Your exact output can be slightly different, as long as the required operations and validations are demonstrated correctly.

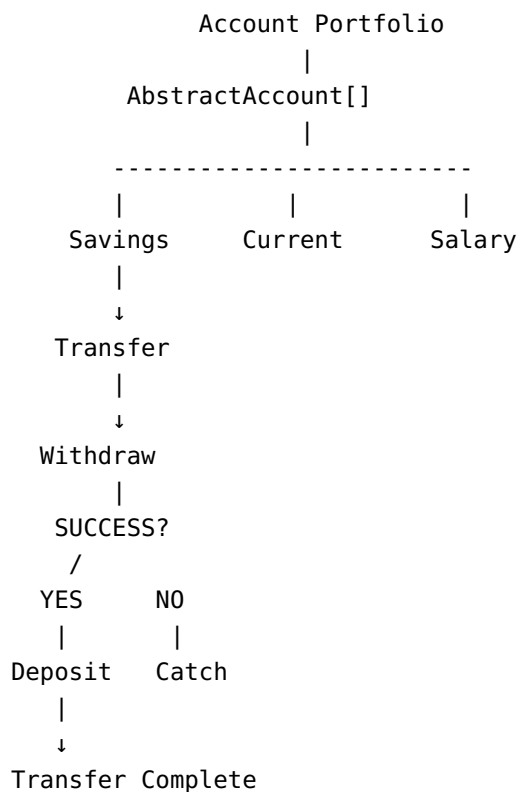
Challenge for You

Try to complete the activity without changing the existing account classes.

Your main job is to build the **banking operations logic** in:

TestAbstractAccount.java

Think of the flow as:



Then process the monthly cycle:

AbstractAccount[]
↓



Learning Outcome

By completing this activity, you should be able to explain:

“A parent-class reference can hold different child-class objects, and we can use polymorphism to work with different account types through a common interface.”

You should also understand how to:

- Work with arrays of abstract-class references
- Perform operations on different child objects
- Handle exceptions during transactions
- Prevent a failed transfer from crediting the destination
- Identify specific account types
- Execute account-specific monthly operations

Try the logic yourself first. Don't worry if you get stuck — focus on understanding the flow before writing the code. 🚀